

EE656 Project

Self-Optimal Clustering Technique Using Optimized Threshold Function

Team Members

1. Karan Chauhan (240519)
2. Muragesh Channappa Nyamagoud (240669)
3. Paveet Saluja (240742)
4. Pushkar Yadav (240816)

Index

Part 1: Basics

Part 1:1. Introduction

Part 1:2. History

Part 1:3. Prerequisites

Part 2: Core Concepts

Part 2:1. Main Analogy – How SOC Works

Part 2:2. Optimization of δ_m and Calculation of β_m

Part 2:3. Need for Optimum Threshold Function

Part 2:4. Key Mathematical Foundations and Validation in the SOC Algorithm

Part 3: Comparison

Part 3:1. Difference Between SOC and Traditional Clustering

Part 3:2. Key Advantages and Benefits of SOC

Part 4: Conclusion

Part 1: Basics

1.1 Introduction

Clustering is a fundamental technique in unsupervised machine learning that aims to group a set of objects in such a way that objects in the same group (called a cluster) are more similar to each other than to those in other groups. Typically, clustering is used in data analysis, pattern recognition, image segmentation, bioinformatics, fraud detection along with other domains.

Clusters are expected to be internally homogeneous and externally heterogeneous meaning that the data points within a cluster should be similar, while clusters themselves should be distinctly different from one another.

Clustering techniques can broadly be categorized as either hard or fuzzy. In hard clustering, each data point belongs strictly to one cluster. In contrast, fuzzy clustering allows partial membership, where each point can belong to multiple clusters with different degrees of association.

Despite the wide range of existing clustering algorithms such as Fuzzy C-Means (FCM), probabilistic clustering, and mountain-based clustering, most struggle to consistently identify all relevant clusters in a dataset due to assumptions about cluster shape or similarity metrics. Following Occam's Razor, a method that relies on fewer assumptions, is generally better and adaptable across various datasets.

To overcome these limitations, the Self-Optimal Clustering (SOC) technique was proposed. SOC introduces a mathematically optimized threshold function that allows better identification of relevant clusters by dynamically adapting to data characteristics. This leads to improved performance in terms of cluster validity, visualization, and efficiency.

1.2 History

Clustering has been a fundamental topic in data analysis and pattern recognition for decades. Over time, numerous clustering algorithms have been proposed to improve accuracy, efficiency, and cluster validity, especially for high-dimensional data.

Early Foundations: The roots of clustering date back to early statistical classification methods. K-means clustering, first proposed by Lloyd in 1957 and refined by MacQueen in 1967, partitions data into k clusters by minimizing within-cluster sum of squares. In the 1970s, Fuzzy C-Means (FCM) clustering was developed by Dunn and further enhanced by Bezdek. FCM allows elements to belong to multiple clusters with varying degrees of membership, introducing the concept of "fuzzy" boundaries rather than rigid partitions.

Grid-Based Techniques: One of the significant developments in clustering was the introduction of Mountain Clustering by Yager and Filev. This method is grid-based and works by identifying potential cluster centers by measuring "mountain functions"

over a grid of data. While intuitive and simple, the method suffers from scalability issues as dimensionality increases. The complexity grows exponentially, which restricts its usage in large or high-dimensional datasets.

Density-Based Approaches: DBSCAN was introduced in 1996 by Ester, Kriegel, Sander, and Xu. DBSCAN can discover clusters of arbitrary shapes and automatically handle noise and outliers without requiring prior specification to number of clusters.

Improved Mountain Clustering (IMC): To address the inefficiency of basic mountain clustering, improved versions like MMC (Modified Mountain Clustering) and IMC (Improved Mountain Clustering) were introduced. IMC determines one cluster at a time, removes it, and recalculates the potential for remaining points. This allows more precise cluster detection and reduces redundancy. It was a significant step toward refining cluster boundaries and improving overall performance.

Segmentation-Based Techniques: In parallel, researchers also explored clustering for image segmentation. Techniques like normalized cuts, saddle point detection, multiresolution clustering, and color-texture region identification gained prominence. These methods aimed at dividing images into meaningful regions by clustering feature vectors in hyperspace. However, many of these techniques did not model underlying physical processes but relied on mathematical representations alone.

Recent Innovations and SOC: Many recent algorithms like Expectation-Maximization, K-means, and K-medoid became popular due to their conceptual simplicity and wide applicability. However, challenges remained in discovering all relevant clusters, handling overlapping data, and optimizing computational efficiency.

The Self-Optimal Clustering (SOC) technique builds on the IMC framework and introduces a mathematically optimized threshold function. Instead of heuristic estimates, SOC employs Lagrange's interpolation polynomial to derive an optimal threshold for cluster center selection. This results in better accuracy, fewer redundant clusters, and stronger performance across various cluster validity metrics.

Technique	Key Contribution
K-means (1957/1967)	Partitioning approach using centroid-based clustering with k clusters.
Fuzzy C-Means (1973)	Introduced fuzzy clustering with degrees of membership.
Mountain Clustering	Grid-based approach for detecting cluster centers.
DBSCAN (1996)	Density-based clustering for arbitrary shapes and noise handling.
MMC / IMC	Improved performance by reducing redundant clusters and handling one cluster at a time.
SOC	Optimizes threshold using Lagrange interpolation; enhances cluster quality.

Table 1: Evolution of Clustering



Figure 1: Preferred Clustering Methods for Various Data Geometries.

1.3 Prerequisites

This section outlines fundamental concepts essential for understanding clustering techniques, with a focus on evaluating cluster quality and determining the optimal number of clusters.

This section outlines fundamental concepts essential for understanding clustering techniques, with a focus on evaluating cluster quality and determining the optimal number of clusters.

A. Measures of Cluster Quality

Evaluating the quality of generated clusters is crucial for assessing the performance of any clustering algorithm. The effectiveness of a clustering technique is often quantified by various validation indices that measure both the compactness within clusters and the separation between different clusters. Ideally, for better clustering quality, intracluster distances (distances among data points within the same cluster) should be minimized, while intercluster distances (distances between different clusters) should be maximized.

This report employs four widely accepted validation indices to measure cluster quality, providing accurate and reliable results:

1. **Global Silhouette Index (GSI):** The GSI assigns a quality measure, known as the silhouette width $s(i)$, to each sample i within a given cluster X_m . The silhouette width indicates the confidence of the i^{th} sample's membership in cluster X_m . It is defined as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (1)$$

where $a(i)$ is the average distance between the i^{th} sample and all other samples in its own cluster X_m , and $b(i)$ is the minimum of the average distances between the i^{th} sample and all samples in any other cluster X_k (where $k \neq m$). The silhouette width $s(i)$ ranges from -1 to 1. A value close to 1 signifies that the sample is well-clustered, meaning it is assigned to an appropriate cluster. A value near zero suggests the sample could also belong to a neighboring cluster.

The silhouette value S_m for the m^{th} cluster is the average silhouette width of all samples in that cluster:

$$S_m = \frac{1}{N_m} \sum_{i=1}^{N_m} s(i) \quad (2)$$

The GSI for a partition with M clusters is then computed as the average of the silhouette values of all clusters:

$$GSI = \frac{1}{M} \sum_{m=1}^M S_m \quad (3)$$

A GSI value closer to unity indicates better cluster formation.

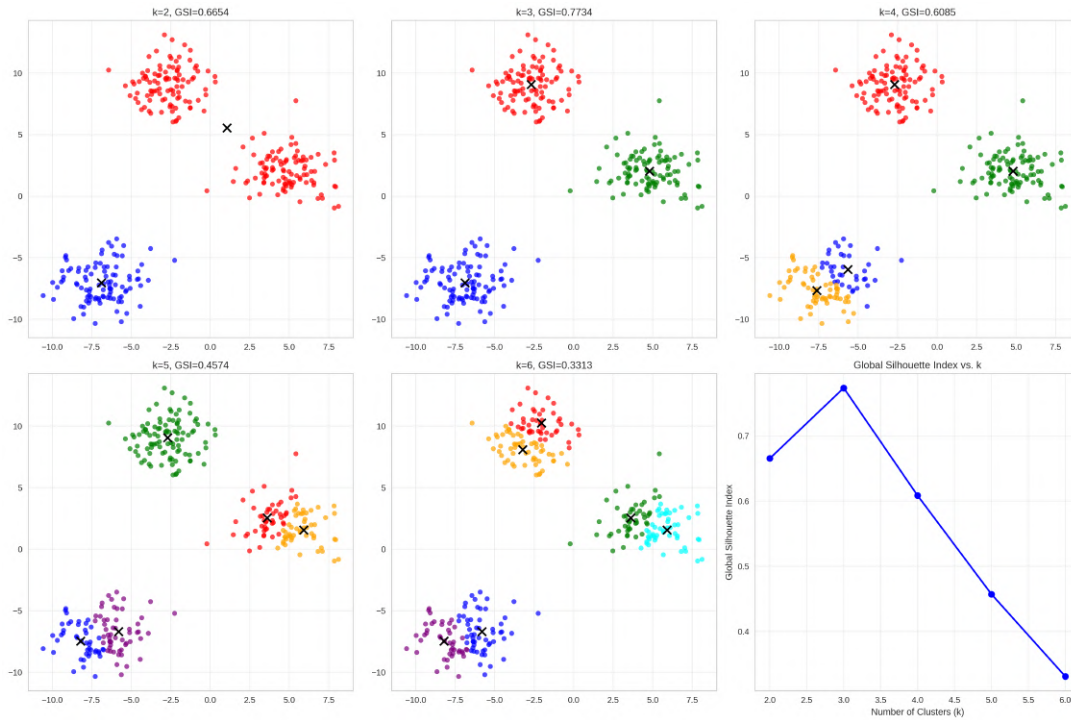


Figure 2: Varying number of clusters over same dataset and measuring their GSI value.

- Partition Index (PI):** The PI is a ratio that quantifies the sum of compactness and separation of clusters. It is calculated by normalizing the sum of individual cluster validity measures by the fuzzy cardinality of each cluster:

$$PI = \frac{\sum_{m=1}^M \sum_{j=1}^n (\mu_{jm})^2 \|\bar{\mathbf{x}}^j - \bar{\mathbf{c}}_m\|^2}{N_m \sum_{k=1}^M \|\bar{\mathbf{c}}_k - \bar{\mathbf{c}}_m\|^2} \quad (4)$$

where $\bar{\mathbf{c}}_m$ is the m^{th} cluster center, N_m is the fuzzy cardinality (sum of memberships μ_{jm}), and μ_{jm} is the membership of data point j in cluster m . A lower value of PI signifies a better partition.

- Separation Index (SI):** The SI measures partition validity based on minimum-distance separation. A lower SI value indicates a more effective partition.

$$SI = \frac{\sum_{m=1}^M \sum_{j=1}^n (\mu_{jm})^2 \|\bar{\mathbf{x}}^j - \bar{\mathbf{c}}_m\|^2}{n \cdot \min_{k,m} \|\bar{\mathbf{c}}_k - \bar{\mathbf{c}}_m\|^2} \quad (5)$$

- Dunn Index (DI):** The DI is designed to identify clusters that are compact and well-separated. For a partition V consisting of clusters $X_1, X_2, \dots, X_M$, the Dunn's validation index is defined as:

$$DI = \min_{1 \leq m \leq M} \left\{ \min_{\substack{1 \leq k \leq M \\ k \neq m}} \left\{ \frac{d(X_m, X_k)}{\max_{1 \leq m \leq M} \{\Delta(X_m)\}} \right\} \right\} \quad (6)$$

where $d(X_m, X_k)$ represents the average centroid linkage intercluster distance between clusters X_m and X_k , and $\Delta(X_m)$ is the complete diameter intracluster distance of cluster X_m . The objective of DI is to maximize intercluster distances while minimizing intracluster distances. Therefore, a larger DI value indicates better cluster quality. The complete diameter intracluster distance $\Delta(X_m)$

is given by $\max_{x,y \in X_m} \{d(x,y)\}$, where $d(x,y)$ is the distance between any two samples x and y within cluster X_m . The centroid linkage intercluster distance is defined as:

$$d(X_m, X_k) = \frac{1}{(|X_m| + |X_k|)} \left(\sum_{x \in X_m} d(x, v_t) + \sum_{y \in X_k} d(y, v_s) \right) \quad (7)$$

where v_s and v_t are centroids of X_m and X_k respectively.

B. Determining Optimum Number of Clusters

Identifying the optimal number of clusters (M) in a dataset is a critical step in clustering analysis, as it directly impacts the meaningfulness and quality of the resulting partition. An incorrect number of clusters can lead to over-segmentation or under-segmentation, hindering effective data interpretation.

Several approaches exist for determining the optimal number of clusters, often relying on validation indices that assess cluster compactness and separation. The Self-Optimal Clustering (SOC) technique itself is designed to be more effective in identifying the optimum number of clusters, with its performance supported by various validity indices. The algorithm's iterative process aims to converge towards a state where the GSI value for the formed clusters is maximized, thereby yielding the best possible cluster structure.

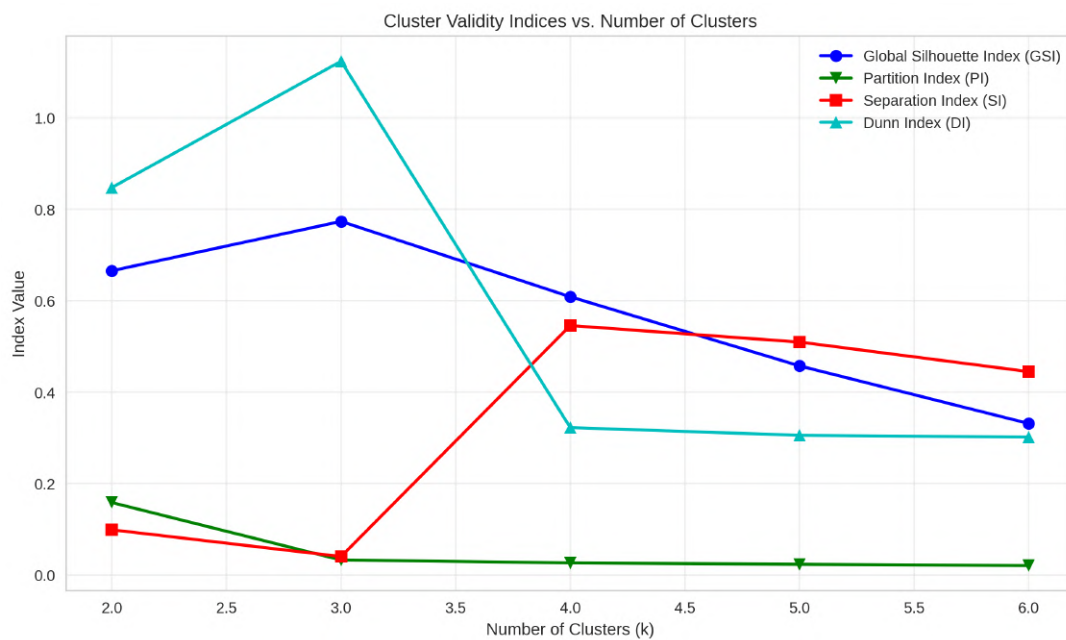


Figure 3: For above dataset, an optimal clustering appears to be achieved with three clusters ($k=3$), as indicated by the simultaneous maximization of the GSI and Dunn Index, and minimization of the PI and SI.

C. Curve Fitting

Curve fitting is a fundamental technique in data analysis that involves finding a mathematical function that best represents the relationship between variables in a dataset. The process can involve either interpolation, where an exact fit to the data is required, or regression, where a function approximately fits the data. This enables data interpretation, trend identification, noise reduction, and predictive modeling across various domains.

While regression aims to model the general trend in potentially noisy data by finding an approximating function, interpolation focuses on constructing a curve that passes precisely through every given data point. This exact-fit approach is particularly vital when the data points are considered highly accurate and represent specific, critical values, or when a precise mapping between points is necessary for internal algorithmic calculations. However, it is highly sensitive to noise, incorporating and propagating it. Interpolation generally exhibits high variance, as small changes in data points significantly alter the curve, and typically has low bias at the interpolation points themselves.

Among interpolation methods, the Lagrange Interpolating Polynomial is widely used due to its direct construction of the unique polynomial of the lowest degree that passes through a given set of data points. For the SOC technique, Lagrange's interpolation method is particularly chosen because the distribution of δ_m in color segmentation often lacks uniformity, necessitating an interpolation approach capable of accurately handling such irregular data points. This characteristic makes it suitable for scenarios where a precise determination of an 'optimizing factor' from specific calculated points is essential for algorithm functionality. Its straightforward computation for a fixed set of points further reinforces it as a fitting choice for such a precise internal calculation within the SOC algorithm.

For $n + 1$ data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, the Lagrange interpolating polynomial $P_n(x)$ is given by:

$$P_n(x) = \sum_{j=0}^n y_j L_j(x)$$

Where $L_j(x)$ are the Lagrange basis polynomials, defined as:

$$L_j(x) = \prod_{k=0, k \neq j}^n \frac{x - x_k}{x_j - x_k}$$

Optimum no. of Clusters = 2	Clustering Method	GSI	PI	SI
	IMC-1	0.8206	0.2271	0.0837
	IMC-2	0.8217	0.2266	0.0836
	SOC	0.8260	0.2222	0.0823

Table 2: Despite the foundational similarity in threshold functions across IMC1, IMC2, and SOC, the observed variations in Global Silhouette Index (GSI), Partition Index (PI), and Separation Index (SI) values are primarily attributable to the optimizing factor. This factor is uniquely obtained through iterative computations via Lagrange interpolation within the SOC algorithm, contributing to its enhanced performance in cluster quality.

Part 2: Core Concepts

2.1 Main Analogy – How SOC Works

The Self-Optimal Clustering (SOC) algorithm improves traditional clustering by adaptively determining an optimized threshold using silhouette analysis and interpolation techniques. Below, we present a step-by-step explanation of how the algorithm works in detail:

Step 1) Normalization of Input Data: To ensure fair comparison across dimensions and prevent scale-based bias, the data is normalized to fit within a unit hypercube. Each data point x_j in D -dimensional space is represented as:

$$\mathbf{x}^j = \{x_1^j, x_2^j, \dots, x_D^j\}$$

It is normalized as:

$$\bar{\mathbf{x}}^j = \frac{\mathbf{x}^j - (\mathbf{x})_{\min}}{(\mathbf{x})_{\max} - (\mathbf{x})_{\min}}, \quad \forall j = 1, 2, \dots, n$$

where $(\mathbf{x})_{\min}$ and $(\mathbf{x})_{\max}$ are vectors containing the minimum and maximum values in each dimension respectively.

Step 2) Threshold Calculation for Neighborhood Definition: A threshold δ_m determines the neighborhood of influence for the m -th cluster. Initially, an optimizing factor $\beta_m = 1$ is assumed. The threshold is computed as:

$$\delta_m = \left(\frac{1}{2n} \sum_{j=1}^n \frac{\min(\mathbf{x}^j)}{\sum_{i=1}^D x_i^j} \right) \cdot \beta_m$$

This ensures the threshold is sensitive to the distribution and density of data points.

Step 3) Computing Potential Using Mountain Function: Each point's potential is computed based on how densely other points are packed around it, using a Gaussian-like mountain function:

$$P_m^r = \sum_{j=1}^n \exp \left(-\frac{d^2(\bar{\mathbf{x}}^r, \bar{\mathbf{x}}^j)}{\delta_m^2} \right)$$

This gives higher values to points that lie in dense regions.

Step 4) Selecting Cluster Center: The data point with the highest potential is selected as the cluster center:

$$\bar{c}_m = \bar{\mathbf{x}}^* \quad \text{where} \quad P_m^* = \max_{1 \leq r \leq n} (P_m^r)$$

This ensures that each cluster is initialized at the most representative (dense) point.

Step 5) Assigning Points to Cluster: Points are assigned to cluster m if their distance from the cluster center is within the threshold δ_m :

$$d^2(\bar{\mathbf{x}}^r, \bar{\mathbf{c}}_m) \leq \delta_m$$

This helps in forming a compact cluster around the chosen center.

Step 6) Removing Assigned Points: Once assigned, the data points are removed from the dataset to avoid being clustered again. This reduces redundancy and ensures new clusters form around unassigned points.

Step 7) Repeat Until All Clusters Are Formed: Repeat Steps 2–6 until the desired number of clusters M is formed. At each iteration, the threshold and cluster center are computed based on the reduced data.

Step 8) Assign Leftover Points: Some points may not fall within any cluster in the initial steps. These points are assigned to the nearest cluster center based on Euclidean distance, ensuring full data utilization.

Step 9) Evaluate Clustering Quality Using Silhouette Index: To evaluate how well-separated and compact the clusters are, the silhouette index S_m is computed for each cluster. The overall clustering quality is measured by the Global Silhouette Index (GSI):

$$\text{GSI} = \frac{1}{M} \sum_{m=1}^M S_m$$

A GSI value closer to 1 indicates better clustering.

Step 10) Interpolating δ vs Silhouette Using Lagrange Polynomial: We relate threshold δ_m and silhouette S_m using Lagrange interpolation:

$$S_t = \sum_{m=1}^M S_m \cdot l_m(\delta_t)$$

where

$$l_m(\delta_t) = \prod_{\substack{k=1 \\ k \neq m}}^M \frac{\delta_t - \delta_k}{\delta_m - \delta_k}$$

This polynomial helps predict silhouette performance for any threshold δ_t .

Step 11) Set Optimal Silhouette Value: Set $S_t = 1$ (ideal value) in the interpolated polynomial to find the optimal δ_t :

$$1 = \sum_{m=1}^M S_m \cdot l_m(\delta_t)$$

Solving this equation gives candidate threshold values that would maximize clustering quality.

Step 12) Select Optimal δ_t : From the roots obtained in the previous step, choose the value of δ_t for which the actual silhouette value is closest to 1. This is our refined threshold, denoted as η .

Step 13) Update Optimization Factor β_m : For each cluster, update the threshold optimization factor:

$$\beta_m = \frac{\eta}{\delta_m}, \quad \forall m = 1, 2, \dots, M$$

This factor will improve the next threshold estimation, guiding the algorithm toward better clusters.

Step 14) Repeat Optimization Until Convergence: With the updated β_m , repeat Steps 2–14. Continue until δ_m values converge and GSI stabilizes. This yields self-optimized clusters.

The detailed iterative refinement of δ_m using silhouette feedback and interpolation is what makes SOC robust and self-adaptive compared to traditional clustering methods.

2.2 Optimization of δ_m and Calculation of β_m

To improve the quality of clusters, the Self-Optimal Clustering (SOC) method adapts the threshold parameter δ_m using a feedback mechanism based on clustering validation indices.

To optimize δ_m , we analyze its impact on inter-cluster and intra-cluster distances, which in turn influence the above indices. In SOC, GSI is used as the primary criterion for improving δ_m .

Using GSI values obtained at each iteration, a Lagrange interpolation method is applied to model the relationship between δ_t and S_t . The interpolation finds the threshold η where GSI is maximum. A corresponding scaling factor β_m is computed as:

$$\beta_m = \frac{\eta}{\delta_m}$$

This β_m is used to adjust δ_m for the next iteration. This process is repeated for a fixed number of iterations (usually 10) until δ_m converges and the maximum GSI is achieved.

Optimum no. of Clusters, M = 2	No. of iterations	δ_m		S_m		$GSI = \left(\frac{\sum_{m=1}^M S_m}{M} \right)$	η	$\beta_m = \frac{\eta}{\delta_m}$	
		δ_1	δ_2	S_1	S_2			β_1	β_2
	1	0.0326	0.0617	0.9093	0.7318	0.8206	0.0177	0.5429	0.2869
	2	0.0177	0.0166	0.9170	0.7285	0.8228	0.0181	1.0226	1.0904
	3	0.0334	0.0677	0.9093	0.7318	0.8206	0.0159	0.4760	0.2349
	4	0.0155	0.0135	0.9170	0.7285	0.8228	0.0164	1.0581	1.2148
	5	0.0344	0.0752	0.9093	0.7318	0.8206	0.0135	0.3924	0.1795
	6	0.0128	0.0101	0.8484	0.7455	0.7970	0.0168	1.3125	1.6634
	7	0.0427	0.1069	0.9154	0.7297	0.8226	0.0135	0.3162	0.1263
	8	0.0103	0.0069	0.8484	0.7455	0.7970	0.0152	1.4758	2.2029
	9	0.0481	0.1412	0.9251	0.7268	0.8260	0.0130	0.2703	0.0921
	10	0.0088	0.0049	0.8484	0.7455	0.7970	0.0144	1.6364	2.9388

Table 3: Progression of δ_m , S_m , η , and β_m over iterations for a two-color image, with the maximum GSI achieved in iteration 9 justifying the selection of δ_m and β_m at this point.

2.3 Need for Optimum Threshold Function

In traditional clustering methods such as IMC-1 and IMC-2, the threshold function plays a crucial role in defining the neighborhood of influence during cluster formation. However, both IMC-1 and IMC-2 exhibit significant limitations when it comes to performance consistency across diverse datasets.

IMC-1 uses a fixed heuristic threshold, which often fails to adapt to varying data distributions, leading to poor clustering compactness and separability. On the other hand, IMC-2 attempts to improve upon IMC-1 by modifying the threshold function heuristically, but this modification lacks a data-driven mechanism and often results in inconsistent behavior. In some cases, IMC-2 even underperforms compared to IMC-1, highlighting the unreliability of its heuristic-based improvements.

Why Optimization is Needed:

- **Heuristic Limitations:** The threshold function in IMC-2 is highly dependent on manual tuning. This introduces subjectivity and lacks a robust mathematical foundation.
- **Quality Instability:** Cluster quality metrics such as the Global Silhouette Index (GSI), Partition Index (PI), and Separation Index (SI) show wide variations in performance, with IMC-1 and IMC-2 sometimes delivering suboptimal or unstable results.
- **Lack of Adaptability:** These methods do not adjust based on cluster quality feedback, which is critical in unsupervised learning scenarios.

SOC's Contribution: SOC addresses the above shortcomings by introducing a systematic and data-adaptive optimization of the threshold function. It leverages the silhouette index — a well-established cluster quality measure — and uses Lagrange interpolation to refine the threshold value for each cluster iteratively. This optimization process:

- Provides a quantitative relation between silhouette score and threshold value.
- Allows automatic adjustment of threshold values to maximize cluster quality.
- Leads to the formation of compact and well-separated clusters with minimal manual intervention.

Optimum no. of Clusters = 2	Clustering Method	GSI
	IMC-1	0.8680
	IMC-max	0.8659
	IMC-half	0.8641
	IMC-2	0.8709
	SOC	0.8765

Table 4: Evidently, SOC demonstrates superior performance compared to the other clustering methods evaluated.

Assessing the Efficacy of SOC Relative to Other Clustering Methods Using Key Clustering Indices.

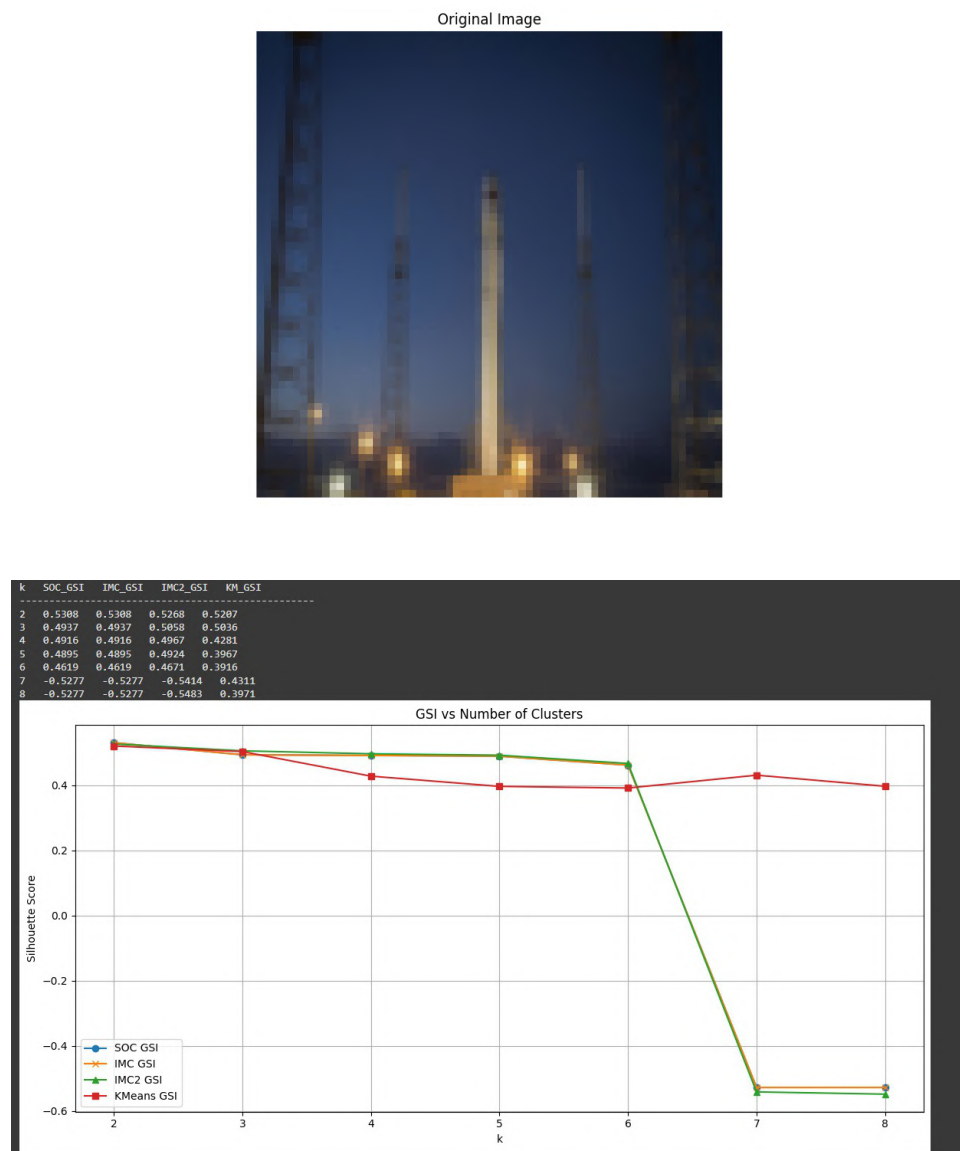


Figure 4: Clearly, SOC algorithm demonstrates superior performance compared to other methods.

2.4 Key Mathematical Foundations and Validation in the SOC Algorithm

This section explains the key mathematical formulas used in the Self-Optimal Clustering (SOC) algorithm. These formulas help compute the threshold values needed for clustering and ensure the algorithm converges properly.

Threshold Function Equations

The threshold value δ_t is calculated using the minimum and total values of features across all data points. The basic form is:

This is refined in the next iteration using a convergence factor:

$$\delta_t = \left(\frac{1}{2n} \sum_{j=1}^n \frac{\min(\mathbf{x}^j)}{\sum_{i=1}^D x_i^j} \right) \cdot \beta_{I-1} \quad \text{where} \quad \beta_{I-1} = \frac{\eta_{I-1}}{\delta_{I-1}}$$

An expanded version involves multiple past iterations:

$$\delta_t = \left(\frac{1}{2n} \sum_{j=1}^n \frac{\min(\mathbf{x}^j)}{\sum_{i=1}^D x_i^j} \right)_I \cdot \left(\frac{\eta_{I-1}}{\left(\frac{1}{2n} \sum_{j=1}^n \frac{\min(\mathbf{x}^j)}{\sum_{i=1}^D x_i^j} \right)_{I-1}} \cdot \beta_{I-2} \right)$$

These recursive expressions help update δ_t using earlier values and control parameters η and β .

Silhouette Score using Lagrange Interpolation

To calculate the silhouette score S_t for a given δ_t , Lagrange interpolation is applied using known pairs (δ_i, S_i) , and is expanded as follows in the full polynomial form:

$$S_t = S_1 \cdot \frac{(\delta_t - \delta_2)(\delta_t - \delta_3) \cdots (\delta_t - \delta_M)}{(\delta_1 - \delta_2)(\delta_1 - \delta_3) \cdots (\delta_1 - \delta_M)} + \cdots + S_M \cdot \frac{(\delta_t - \delta_1)(\delta_t - \delta_2) \cdots (\delta_t - \delta_{M-1})}{(\delta_M - \delta_1)(\delta_M - \delta_2) \cdots (\delta_M - \delta_{M-1})}$$

This equation ensures that S_t exactly matches known silhouette values at corresponding threshold points.

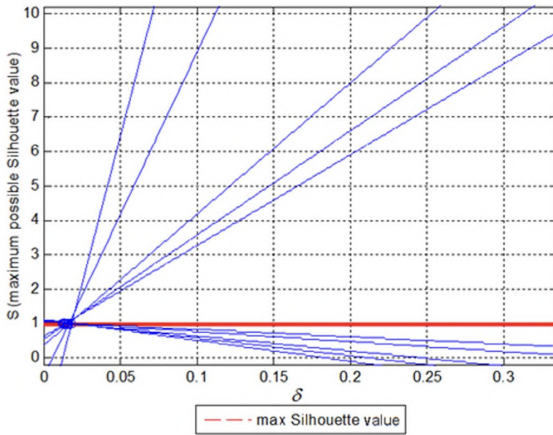


Figure 5: Variation of S with threshold value δ over the range $[0, 0.3333]$ (3 Features)

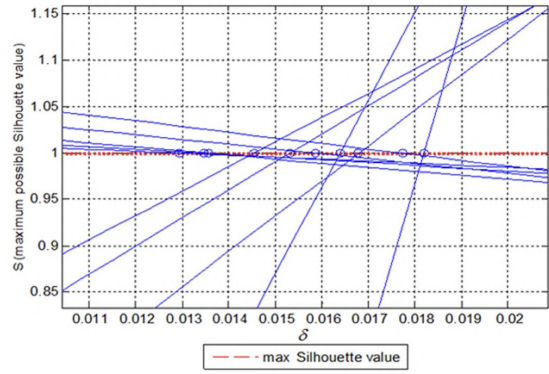


Figure 6: Computation of interpolation nodes η using function $S_t = \text{const.} \cdot f(\delta_t)$

Above shows the behavior of the silhouette score S_t for different threshold values δ_t :

- The left plot shows variation of S with δ using interpolation.
- The right plot shows values of S_t across multiple iterations and cluster settings.

From the plots, it is observed that the values of δ_t where interpolation curves intersect max silhouette value, give the most consistent and optimal silhouette values. This confirms that the threshold function converges and supports optimal clustering results.

Part 3: Comparison

3.1 Difference Between SOC and Traditional Clustering

Various experiments were performed on diverse image datasets to assess SOC's effectiveness relative to standard clustering algorithms. RGB features were utilized in these analyses, with cluster quality evaluated using key metrics such as GSI, SI, PI, and DI, as demonstrated by the following examples:

Optimum no. of Clusters = 4	Clustering Method	GSI	PI	SI	DI
	K-means	0.6986	0.0541	0.0127	0.5255
	FCM	0.6967	0.0515	0.0119	0.5354
	EM	0.3680	0.0950	0.0535	0.0399
	K-medoid	0.6941	0.0547	0.0129	0.4965
	IMC-1	0.7037	0.0551	0.0130	0.5650
	IMC-2	0.6984	0.0623	0.0152	0.5269
	SOC	0.7056	0.0527	0.0119	0.6443

Table 5: A bottle illuminated by background lighting

Optimum no. of Clusters = 2	Clustering Method	GSI	PI	SI	DI
	K-means	0.8735	0.1006	0.0497	0.9921
	FCM	0.8741	0.0940	0.0465	0.9502
	EM	0.8716	0.1013	0.0499	0.9987
	K-medoid	0.8727	0.0972	0.0480	0.9913
	IMC-1	0.8746	0.0840	0.0416	0.9936
	IMC-2	0.8731	0.0864	0.0427	0.9917
	SOC	0.8747	0.0867	0.0431	0.9940

Table 6: Image depicting a flower

3.2 Key Advantages and Benefits of SOC

The Self-Optimal Clustering (SOC) algorithm offers significant advantages over traditional and intermediate clustering techniques, particularly due to its adaptive threshold optimization and iterative silhouette-based feedback mechanism. These benefits are highlighted through both qualitative properties and quantitative performance metrics, and are summarized in Table 6. SOC not only improves cluster separability and compactness but also adapts intelligently to the data through interpolation-driven threshold optimization. This makes it especially suited for applications involving noisy, high-dimensional, or complex datasets where conventional methods struggle.

Feature	K-means	IMC-1	IMC-2	SOC
Handles Non-linearity	No	Yes	Yes	Yes
Adaptive Threshold	No	No	Partial	Yes
Uses Silhouette Feedback	No	No	No	Yes
Data-driven Optimization	No	No	No	Yes
Consistent Cluster Quality	Medium	Medium	Varies	High

Table 7: Method-wise qualitative comparison

The quantitative advantages of SOC are clearly demonstrated in Table 2, where its superior performance is evident. This effectiveness is directly attributable to its optimized threshold function.

Actual /Assumed name	Description of changes made	Modified threshold function
IMC-1	IMC technique with no changes in the threshold function	$\delta_m = \left(\frac{1}{2n} \sum_{j=1}^n \frac{\min(x^j)}{\sum_{i=1}^D x_i^j} \right)$
IMC-max	IMC technique with 'min' replaced by 'max' in the threshold function	$\delta_m = \left(\frac{1}{2n} \sum_{j=1}^n \frac{\max(x^j)}{\sum_{i=1}^D x_i^j} \right)$
IMC-half	IMC technique with the factor of '1/2' removed from the threshold function	$\delta_m = \left(\frac{1}{n} \sum_{j=1}^n \frac{\min(x^j)}{\sum_{i=1}^D x_i^j} \right)$
IMC-2	IMC technique with a factor 'no. of cluster / (no. of cluster + 1)' multiplied to the threshold function	$\delta_m = \left(\frac{1}{2n} \sum_{j=1}^n \frac{\min(x^j)}{\sum_{i=1}^D x_i^j} \right) \left(\frac{nk}{nk+1} \right)$ where nk = total number of clusters
SOC	Proposed algorithm using IMC technique with no inherent changes in the INITIAL threshold function and a factor is multiplied in later threshold functions EXTERNALLY only	$\delta_m = \left(\frac{1}{2n} \sum_{j=1}^n \frac{\min(x^j)}{\sum_{i=1}^D x_i^j} \right) (\beta_m)$ where β_m = optimizing factor multiplied externally

Table 8: Overview of Threshold Function Used in Different Clustering Algorithms

Part 4: Conclusion

This paper introduces Self-Optimal Clustering (SOC), an advanced mountain clustering technique that optimizes its threshold function using Lagrange's interpolation. Unlike other common methods like K-means and Fuzzy C-means, which often struggle with optimal cluster formation or parameter sensitivity, SOC consistently demonstrates superior cluster quality. This is validated by maximizing the Global Silhouette Index (GSI) and various standard indices, showcasing its enhanced compactness and separation. Future work will explore SOC's application to diverse real-world datasets and investigate its scalability for large-scale analysis, highlighting its significant promise in advancing data clustering.